

ADVERTEX AI STUDIO

BRIEFING MESTRE DE PRODUTO E ENGENHARIA

Plataforma desktop para criação, gestão, análise e otimização de anúncios com IA — Meta Ads e Google Ads.

Objetivo: orientar o Claude Code a construir uma aplicação real, modular, segura e instalável no Windows, com atalho na área de trabalho.

Documento de referência para desenvolvimento incremental. Recursos que dependem de APIs, credenciais, aprovações ou serviços externos devem ser identificados como dependências — nunca simulados como concluídos.

Versão do briefing: 1.0 • Data: outubro de 2026

Sumário executivo

O ADVERTEX AI Studio é uma plataforma de marketing orientada por IA, concebida para reunir briefing estratégico, criação de anúncios, gestão de ativos, integração com plataformas publicitárias, análise de desempenho e automação auditável em uma única experiência desktop.

O diferencial do produto não é apenas gerar textos. É fechar o ciclo entre entender o negócio, criar campanhas, publicar por APIs oficiais, medir resultados reais, documentar experimentos e transformar aprendizados em recomendações futuras.

Regra de produto: diferenciar claramente criação, publicação e otimização. Não apresentar dados fictícios como reais nem declarar uma publicação concluída sem confirmação da plataforma.

1. Visão e objetivos

- Aplicativo desktop Windows com Electron, instalador EXE, atalho na área de trabalho e menu Iniciar.
- Gestão unificada de projetos, marcas, criativos, campanhas e contas publicitárias Meta Ads e Google Ads.
- Criação manual e assistida por IA de textos, imagens, variações e estruturas de campanha.
- Leitura de dados reais, análise de desempenho e recomendações rastreáveis.
- Automações configuráveis com simulação, aprovação humana, limites de gasto e auditoria.
- Arquitetura modular que possa evoluir de uso local para backend hospedado e multiusuário.

2. Funcionalidades e diferenciais

2.1 Inteligência de negócio e briefing

- Cadastro do segmento, produto, serviço, oferta, região, idioma, público, diferenciais, objeções, tom de voz e objetivos.
- Análise de páginas publicamente acessíveis, com controles contra SSRF e sem contornar autenticação.
- Geração de resumo do negócio, proposta de valor, pilares de comunicação, hipóteses de campanha, matriz de mensagens e informações ausentes.
- Briefing editável, versionado e associado ao projeto de origem.

2.2 Estúdio criativo com IA

- Geração de textos principais, títulos, descrições, CTAs, roteiros, anúncios de pesquisa e variações por público e etapa do funil.
- Geração de conceitos visuais, upload, recorte, redimensionamento, adaptação de formatos, identidade de marca e exportação.
- Biblioteca de ativos organizada por organização, cliente, projeto, campanha e versão.
- Editor manual, histórico de versões, comparação antes/depois e aprovação de criativos.
- Validação de limites e formatos conforme as regras atuais de cada plataforma.

2.3 Inteligência competitiva

- Cadastro de concorrentes, análise de sites e ofertas públicas, com URL de origem e data de captura.
- Classificação de referências por promessa, conceito, público, formato e posicionamento.
- Identificação de padrões e oportunidades de diferenciação, sem alegar acesso a campanhas privadas.

2.4 Análise de performance

- Painel com investimento, impressões, alcance, cliques, CTR, CPM, CPC, conversões, CPA, receita atribuída e ROAS quando disponíveis.

- Comparação entre períodos, plataformas, campanhas e criativos.
- Detecção de anomalias, gasto excessivo, queda de conversões, possível fadiga criativa e problemas de rastreamento.
- Recomendações com evidências, período analisado, confiança, impacto potencial, riscos e limitações.

2.5 Experimentos

- Registro de hipótese, variável testada, métrica primária, criativos comparados, período e critério de decisão.
- Comparação de variações, documentação de resultados e histórico de aprendizados.
- Indicação explícita de resultados inconclusivos quando volume ou qualidade de dados forem insuficientes.

2.6 Automações

- Regras de gatilho, condições, janela de avaliação, frequência, ação, limites, expiração e aprovação.
- Alertas de gasto, CPA, conversões e mudanças inesperadas.
- Modos: somente leitura, recomendações, execução com aprovação e execução automática limitada.
- Limites máximos de gasto, idempotência, verificação de estado, retentativas controladas, auditoria e botão de emergência.

3. Fluxo de melhoria contínua

Etapa	Responsabilidade
1. Entender	Reunir contexto do negócio, público, oferta, objetivos e restrições.
2. Criar	Gerar estratégia, textos, conceitos visuais e variações.
3. Validar e publicar	Validar campos, políticas, orçamento, permissões e aprovações; publicar pela API oficial.
4. Medir	Importar e analisar métricas reais, preservando origem, período, moeda e definições.
5. Aprender	Registrar resultados, atualizar hipóteses e sugerir próximos experimentos.

4. Arquitetura técnica recomendada

Camada	Tecnologias e responsabilidade
Desktop	Electron, React, TypeScript, Vite; janela, navegação, IPC restrito e experiência local.
UI	Tailwind CSS, shadcn/ui, Lucide, React Router, TanStack Query, React Hook Form, Zod e Recharts.
Backend	Node.js, TypeScript, NestJS, REST/OpenAPI, autenticação, permissões e regras de negócio.
Persistência	PostgreSQL, Prisma ORM, migrações versionadas e índices.
Filas e cache	Redis e BullMQ quando houver tarefas assíncronas e agendamentos.
IA	Adaptadores independentes para provedores de texto, imagem e outros recursos.
Testes	Vitest, React Testing Library, Playwright e testes de contrato com mocks.
Empacotamento	electron-builder ou Electron Forge; EXE, atalhos, desinstalação e atualização futura.

5. Arquitetura do repositório

```
apps/  
  desktop/ (main, preload, renderer)  
  api/ (auth, organizations, projects, integrations, campaigns, creatives, analytics, intelligence,  
  automations, reports)  
packages/  
  shared/ • schemas/ • ui/ • advertising-core/ • ai-core/ • platform-meta/ • platform-google/  
infra/  
  docker/ • scripts/  
docs/  
  architecture/ • setup/ • integrations/ • security/ • testing/
```

A estrutura é uma referência, não uma obrigação. Ajuste-a com justificativa técnica documentada. Não concentre toda a lógica de negócio no processo principal do Electron.

6. Segurança e privacidade

- Electron com contextIsolation, preload restrito, IPC validado e sandbox quando compatível; Node.js desabilitado no renderer.
- Fluxos OAuth adequados para desktop e PKCE quando aplicável; revogação e renovação de tokens.
- Segredos fora do frontend, do bundle, dos logs e do repositório; armazenamento seguro e criptografia de credenciais persistidas.
- Validação de entradas, autorização no backend, rate limiting, proteção contra XSS e SSRF e tratamento seguro de arquivos.
- Isolamento de dados por organização em todas as operações do backend, não apenas por filtros da interface.
- Auditoria de ações críticas, política de retenção, logs sem segredos e revisão de permissões.

7. Integração Meta Ads

Implementar adaptador separado para a Meta Marketing API oficial, com autorização, seleção de contas, leitura de campanhas, conjuntos, anúncios e insights; upload de ativos; criação e edição de entidades compatíveis; pausa, ativação, sincronização, paginação, limites de requisições e registro de operações.

Verificar a versão atual da API, permissões exigidas, revisão do aplicativo e limites de acesso. Armazenar IDs remotos e mapear entidades locais. Nunca declarar sucesso antes da confirmação da API.

Documentação: [Meta Marketing API](#).

8. Integração Google Ads

Implementar adaptador separado para a Google Ads API oficial, com OAuth 2.0, seleção de contas autorizadas, consulta de campanhas e métricas, criação de campanhas e grupos de anúncios, anúncios e recursos compatíveis, alteração de status, sincronização e tratamento de erros.

Validar requisitos atuais de projeto Google Cloud, developer token, nível de acesso, políticas e limites. Não presumir equivalência entre os modelos de campanha do Google Ads e do Meta Ads.

Documentação: [Google Ads API — início](#).

9. Modelo unificado de campanhas

Criar um modelo de domínio interno para campanhas, grupos de anúncios, anúncios, criativos, ativos, orçamento e métricas, preservando diferenças específicas entre plataformas. Cada entidade deve registrar ID interno, plataforma, ID remoto, conta, projeto, estado de sincronização, última sincronização e erros recentes.

- Operações explícitas: listCampaigns, getCampaign, createCampaign, updateCampaign, pauseCampaign, resumeCampaign, fetchInsights, uploadAsset e createAd.
- Contratos e validações independentes para cada plataforma.
- Idempotência e verificação do estado remoto antes de repetir operações de criação.
- Métricas com origem, período, definição e moeda; evitar somas de indicadores incompatíveis.

10. Banco de dados

Projetar schema relacional e migrações Prisma para: User, Organization, Membership, Client, Project, BrandProfile, Brief, BriefVersion, IntegrationConnection, AdvertisingAccount, Campaign, AdGroup, Ad, Creative, Asset, MetricSnapshot, Insight, Recommendation, Experiment, AutomationRule, AutomationExecution, Approval, AuditLog, Report, Notification e AIJob.

Incluir relações, timestamps, índices, restrições de integridade e estratégia de exclusão. Seeds devem ser identificados como demonstração e permanecer separados de dados reais.

11. Relatórios e calendário

- Relatórios por cliente, projeto, plataforma, campanha e período, com resumo executivo, métricas, gráficos, recomendações, alertas e limitações dos dados.
- Exportação CSV e PDF com paginação e layout profissional.
- Calendário de campanhas, lançamentos, tarefas, responsáveis, prazos e aprovações.
- Status agendado não significa publicação concluída; aguardar confirmação da plataforma.

12. Observabilidade e confiabilidade

- Logs estruturados, IDs de correlação, estados de tarefas e mensagens de erro compreensíveis.
- Timeouts, paginação, tentativas com backoff, limites de API, circuit breaker e registro de falhas.
- Não repetir uma criação de campanha sem verificar o resultado da operação anterior.
- Registrar solicitação, resultado, ID remoto e status das operações críticas.

13. Experiência de primeira execução

1. Apresentar boas-vindas e criar organização.
2. Criar o primeiro projeto e preencher o briefing.
3. Oferecer conexão com Meta Ads e Google Ads.
4. Configurar provedor de IA e explicar as credenciais necessárias.
5. Permitir explorar o modo de demonstração, claramente identificado.
6. Mostrar checklist de configuração e permitir trabalhar localmente mesmo sem integrações.

14. Instalador e atalhos no Windows

- Nome: ADVERTEX AI Studio; executável, ícone, versão e identificador estável.
- Gerar instalador EXE, atalho na área de trabalho, atalho no menu Iniciar e desinstalador.
- Usar caminhos corretos para arquivos do usuário e logs de diagnóstico.
- Testar instalação limpa, abertura pelo atalho, reinicialização, atualização e desinstalação.
- Validar o empacotamento em ambiente Windows. Compilar não é suficiente para provar que o atalho foi criado.

Documentação de referência: [electron-builder](#).

15. Estrutura de telas

Tela	Escopo funcional
Visão geral	KPIs, gráficos, alertas, comparação de períodos e filtros.
Projetos	Cadastro, briefing, versões, ativos e objetivos.
Estúdio de IA	Geração de texto, conceitos visuais, edição e aprovação.
Criativos	Biblioteca, versões, formatos, tags e busca.
Campanhas	Campanhas unificadas, estado, orçamento e filtros.
Meta Ads / Google Ads	Contas conectadas, sincronização, status, erros e permissões.
Inteligência	Diagnósticos, evidências, confiança, impacto e ações sugeridas.
Experimentos	Hipóteses, variantes, métricas e conclusões.
Automações	Editor de regras, simulação, aprovações e auditoria.
Relatórios	Criação, exportação, histórico e compartilhamento controlado.
Integrações	Conexões, credenciais, permissões e estado de sincronização.
Configurações	Organização, usuários, segurança, provedores e preferências.

16. Critérios de aceite

1. O aplicativo inicia sem erros e possui navegação funcional.
2. O usuário consegue criar, editar e persistir projetos e briefings.
3. A geração de conteúdo funciona com um provedor configurado.
4. Arquivos podem ser enviados, organizados e recuperados.
5. Métricas de demonstração são claramente identificadas como fictícias.
6. Conectores explicam credenciais, permissões e erros sem simular conexão.
7. Publicação possui validação, confirmação e tratamento de falhas.
8. Telas e fluxos principais não contêm botões fictícios.
9. Testes automatizados passam e os principais cenários estão cobertos.
10. O aplicativo pode ser empacotado para Windows.
11. O instalador cria atalho funcional na área de trabalho e menu Iniciar.
12. A aplicação instalada abre sem depender da pasta de desenvolvimento.
13. Outra pessoa consegue configurar e executar o projeto seguindo a documentação.

17. Plano de implementação por fases

Fase	Entrega
0 — Auditoria	Inspecionar repositório, mapear riscos, arquitetura, backlog e critérios de aceite.
1 — Fundação	Electron, React, TypeScript, navegação, design system, logs, testes e empacotamento inicial.
2 — Projetos	Organizações, projetos, briefing, persistência, edição e histórico.
3 — Estúdio criativo	Geração de texto, biblioteca de ativos, upload, editor e provedores de IA.
4 — Dashboard	Métricas, gráficos, filtros, relatórios e modo de demonstração.
5 — Integrações	Leitura e sincronização; depois criação e publicação controladas.
6 — Inteligência	Diagnósticos, recomendações, comparação criativa e experimentos.
7 — Automações	Regras, simulação, aprovação, auditoria e execução limitada.
8 — Produção	Segurança, regressão, desempenho, documentação e teste de instalação limpa.

18. Instruções de trabalho para Claude Code

1. Inspeção o repositório e a documentação existente antes de alterar arquivos.
2. Se estiver vazio, crie a estrutura inicial; se já houver código, preserve o que funciona.
3. Documente decisões arquiteturais importantes e riscos.
4. Implemente em módulos pequenos, com responsabilidades claras.
5. Após cada fase, execute testes, lint e verificação de tipos; corrija falhas antes de avançar.
6. Quando faltarem credenciais externas, implemente fluxos reais com configuração explícita e testes com mocks; documente o bloqueio e continue o restante.
7. Não declarar uma funcionalidade concluída sem testar seu fluxo.
8. Atualize README.md, docs/ARCHITECTURE.md, docs/ROADMAP.md, docs/SETUP.md, docs/INTEGRATIONS.md, docs/SECURITY.md e docs/TESTING.md.
9. Não parar apenas na apresentação de um plano: após a auditoria, implementar a primeira fase executável.

19. Primeira entrega obrigatória

Entregar uma aplicação Windows executável com identidade visual premium, dashboard, gestão de projetos, formulário completo de briefing, estúdio inicial de criação, biblioteca de criativos, tela de campanhas, painel de integrações, configurações, banco de dados, estados de carregamento/erro/vazio, testes, instalador EXE e atalho funcional na área de trabalho.

Implementar tudo o que não dependa de credenciais externas. Para funcionalidades dependentes de autorização, construir integração configurável e modo de demonstração separado.

20. Definição de sucesso

O produto estará evoluindo corretamente quando um profissional puder criar um projeto, gerar e editar criativos, conectar contas publicitárias, consultar dados reais, preparar campanhas, publicar mediante autorização e receber recomendações rastreáveis. O sistema deve priorizar confiabilidade, segurança, transparência dos dados e melhoria contínua.

COMANDO FINAL AO CLAUDE CODE: Comece agora pela Fase 0, inspecione o repositório e implemente a primeira fase executável. Não se limite a explicar como o projeto poderia ser feito.

Apêndice A — Comandos sugeridos

```
npm run dev  
npm run lint  
npm run typecheck  
npm run test  
npm run build  
npm run dist:win
```

Adapte os comandos à estrutura final do monorepo. Documente pré-requisitos, versões de runtime e variáveis de ambiente no README.

Apêndice B — Checklist rápido antes de produção

- Credenciais e permissões reais configuradas e testadas.
- Dados reais claramente separados de demonstrações.
- Isolamento entre organizações validado.
- Logs revisados para garantir ausência de segredos.
- Operações de publicação e orçamento com auditoria e limites.
- Testes automatizados e teste manual de fluxos críticos concluídos.
- Instalador verificado em Windows limpo.
- Atalho da área de trabalho testado após a instalação.
- Documentação de instalação, backup, recuperação e suporte concluída.